# ConfigMap & Secret — Lab

Env Vars · Volume Mounts · Combined Pattern

⏱ 50 minutes  Д 100 points  📋 7 tasks  Ⓡ CKA Level

## Lab Rules

✓ Use kubectl and YAML files only
✓ Default namespace unless stated otherwise
✓ Apply the ConfigMap and Secret BEFORE creating any Pod
✓ Verify every task with kubectl exec or kubectl logs

# Part 1 — ConfigMap

50 pts

## 1 Create a ConfigMap — Imperative

Easy ☆

Create a ConfigMap named **app-config** using the imperative method — no YAML file needed.

| Field | Value |
|---|---|
| ConfigMap Name | `app-config` |
| Key 1 | `APP_ENV = production` |
| Key 2 | `APP_PORT = 8080` |
| Key 3 | `APP_COLOR = blue` |
| Key 4 | `MAX_CONNECTIONS = 100` |

**Hint:**
kubectl create configmap <name> --from-literal=KEY=value --from-literal=KEY2=value2
You can chain multiple --from-literal flags in one command.

### Requirements:

**a** Create the ConfigMap using `--from-literal` for all 4 keys

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_PORT=8080 \
  --from-literal=APP_COLOR=blue \
  --from-literal=MAX_CONNECTIONS=100
configmap/app-config created
```

**b** Run `kubectl describe configmap app-config` and confirm all 4 keys appear

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
blue

APP_ENV:
----
production

APP_PORT:
----
8080

MAX_CONNECTIONS:
----
100


BinaryData
====
```

**c** Run `kubectl get configmap app-config -o yaml` — what does the output look like?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl get configmap app-config -o yaml
apiVersion: v1
data:
  APP_COLOR: blue
  APP_ENV: production
  APP_PORT: "8080"
  MAX_CONNECTIONS: "100"
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-15T08:10:52Z"
  name: app-config
  namespace: default
  resourceVersion: "144042"
  uid: f6558981-0711-4aef-977c-5b4652f1a1f8
```

**d** What is the difference between a ConfigMap and a regular variable you hardcode in a Pod?

✅ **Verify with:**

```
kubectl get configmap
kubectl describe configmap app-config
kubectl get configmap app-config -o yaml
```

✍ConfigMap allows separating configuration from the application code or Pod specification.

Instead of hardcoding values inside the Pod YAML, we reused by multiple Pods store them in a ConfigMap so they can be updated independently and

## 2 Inject ConfigMap as Environment Variables

**Easy** ☆

Create a Pod that injects **all keys** from the ConfigMap as environment variables using `envFrom`.

| Field | Value |
|---|---|

## % ConfigMap & Secret — Lab

Session 6 · CKA Level

| | |
|---|---|
| File to apply | `02-configmap-as-envvars.yaml` |
| Pod Name | `env-pod` |
| Image | `busybox` |
| Injection method | `envFrom → configMapRef: app-config` |
| Command | `env && sleep 3600` |

### Requirements:

**a** Apply `02-configmap-as-envvars.yaml`

```
u1@ubuntu-desktop1:~/k8s_labs/config$ nano 02-configmap-as-envvars.yaml
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl apply -f 02-configmap-as-envvars.yaml
pod/env-pod created
```

**b** Run `kubectl logs env-pod` and find the 4 ConfigMap keys in the output

```
API_SVC_PORT_80_TCP_ADDR=10.102.55.28
BACKEND3_SERVICE_SERVICE_PORT=80
BACKEND3_SERVICE_PORT=tcp://10.101.60.30:80
WEBAPP_SVC_PORT_80_TCP_PROTO=tcp
APP_COLOR=blue
NGINX_SERVICE_PORT=tcp://10.96.87.252:80
NGINX_SERVICE_SERVICE_PORT=80
API_SVC_PORT_80_TCP_PORT=80
API_SVC_PORT_80_TCP_PROTO=tcp
BACKEND1_SERVICE_PORT_80_TCP_ADDR=10.98.127.57
```

**c** Run `kubectl exec -it env-pod -- env | grep APP` — what do you see?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl exec -it env-pod -- env | grep APP
E0315 11:17:52.891352  542221 runtime.go:142] "Observed a panic" panic="runtime error: invalid memory address or nil pointer dereference" panicGoValue="\"invalid memory address or nil po
inter dereference\"" stacktrace=<
                                        goroutine 134 [running]:
                                                                k8s.io/apimachinery/pkg/util/runtime.logPanic({0x289ca38, 0x3c0f0a0}, {0x2008260, 0x3bb6560})
                                                                                                                                                                k8s.io/api
machinery/pkg/util/runtime/runtime.go:132 +0xbc
                                                k8s.io/apimachinery/pkg/util/runtime.handleCrash({0x289ca38, 0x3c0f0a0}, {0x2008260, 0x3bb6560}, {0xc00065fda8, 0x0, 0x0?})
                                                                                                                                                                k8
s.io/apimachinery/pkg/util/runtime/runtime.go:107 +0x116
                                                                k8s.io/apimachinery/pkg/util/runtime.HandleCrash({0x0, 0x0, 0xc000103c00?})
                                                                                                                                                k8s.io/apimachinery/pkg/util/runti
me/runtime.go:64 +0x17b
                                        panic({0x2008260?, 0x3bb6560?})
                                                                runtime/panic.go:783 +0x132
                                                                                                k8s.io/kubectl/pkg/cmd/exec.(*terminalSizeQueueAdapter).Next(0x0?)
```

**d** Explain: what does `envFrom` do differently than writing each `env:` key manually?

**Verify with:**

```
kubectl logs env-pod
kubectl exec -it env-pod -- env | grep APP
kubectl exec -it env-pod -- env | grep MAX
```

What did you see from grep APP?

The APP variables from the ConfigMap appeared as environment variables inside the containe

Difference between envFrom and individual env: keys:
envFrom imports all keys from a ConfigMap as environment variables automatically, while env requires defining each variable individually

## 3 Inject Specific Keys with Custom Names

**Medium** ☆☆

Create a Pod that injects **specific keys** from the ConfigMap with **custom variable names** inside the container.

| Field | Value |
|---|---|
| File to apply | `03-configmap-selective-env.yaml` |
| Pod Name | `selective-pod` |
| Image | `busybox` |
| Env var name | `COLOR` ← `from APP_COLOR` |
| Env var name | `PORT` ← `from APP_PORT` |

**Note:** The variable name inside the container can be different from the ConfigMap key name.

### Requirements:

## % ConfigMap & Secret — Lab



**a** Apply `03-configmap-selective-env.yaml`

```
u1@ubuntu-desktop1:~/k8s_labs/config$ nano 03-configmap-selective-env.yaml
u1@ubuntu-desktop1:~/k8s_labs/config$
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl apply -f 03-configmap-selective-env.yaml
pod/selective-pod created
```

**b** Check logs — the pod prints `Color=blue` and `Port=8080`

```
BACKEND2_SERVICE_SERVICE_PORT=80
API_SVC_PORT_80_TCP_ADDR=10.102.55.28
BACKEND3_SERVICE_SERVICE_PORT=80
BACKEND3_SERVICE_PORT=tcp://10.101.60.30:80
WEBAPP_SVC_PORT_80_TCP_PROTO=tcp
APP_COLOR=blue
NGINX_SERVICE_PORT=tcp://10.96.87.252:80
NGINX_SERVICE_SERVICE_PORT=80
API_SVC_PORT_80_TCP_PORT=80
API_SVC_PORT_80_TCP_PROTO=tcp
BACKEND1_SERVICE_PORT_80_TCP_ADDR=10.98.127.57
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_SERVICE_PORT_HTTPS=443
BACKEND2_SERVICE_PORT_80_TCP_ADDR=10.110.181.100
BACKEND3_SERVICE_PORT_80_TCP_ADDR=10.101.60.30
```

**c** Run `kubectl exec -it selective-pod -- env` — is `MAX_CONNECTIONS` visible? Why not?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl exec -it selective-pod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=selective-pod
TERM=xterm
COLOR=blue
PORT=8080
ADMIN_SVC_PORT_80_TCP_PROTO=tcp
BACKEND2_SERVICE_PORT_80_TCP_PROTO=tcp
BACKEND3_SERVICE_SERVICE_HOST=10.101.60.30
BACKEND3_SERVICE_SERVICE_PORT=80
API_SVC_SERVICE_HOST=10.102.55.28
API_SVC_PORT_80_TCP_ADDR=10.102.55.28
ADMIN_SVC_SERVICE_PORT=80
```

**No, because only the specific keys defined in the env section are injected into the container**

**d** When would you use selective injection instead of `envFrom`?

**Selective injection is used when the container needs only specific configuration values instead of importing the entire ConfigMap**

**✅ Verify with:**

```
kubectl logs selective-pod
kubectl exec -it selective-pod -- env
```

✍ Is MAX_CONNECTIONS visible? Why not?

When to use selective injection instead of envFrom:

# Part 2 — Secret

30 pts

## 4 Create a Secret and Inspect It

Easy ☆

Create a Secret named **db-secret** to store database credentials.

| Field | Value |
|---|---|
| Secret Name | `db-secret` |
| Type | `Opaque (generic)` |
| Key 1 | `DB_USER = admin` |
| Key 2 | `DB_PASSWORD = p@ssw0rd123` |
| Key 3 | `DB_NAME = myapp` |

**Hint:**
Use the imperative method — kubectl handles base64 encoding for you:
kubectl create secret generic <name> --from-literal=KEY=value

If using YAML (05-secret-basic.yaml), values must be pre-encoded:
echo -n "admin" | base64 → YWRtaW4=

### Requirements:

**a** Create the Secret using the imperative method

**b** Run `kubectl describe secret db-secret` — are the values visible?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl create secret generic db-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=p@ssw0rd123 \
  --from-literal=DB_NAME=myapp
secret/db-secret created
```

**c** Run `kubectl get secret db-secret -o yaml` — how are the values stored?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl get secret db-secret -o yaml
apiVersion: v1
data:
  DB_NAME: bXlhcHA=
  DB_PASSWORD: cEBzc3cwcmQxMjM=
  DB_USER: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-03-15T10:02:10Z"
  name: db-secret
  namespace: default
  resourceVersion: "150321"
  uid: 5bfd5340-f4f5-4faf-8b98-13d4c520f6ce
type: Opaque
```

**d** Decode DB_USER: `kubectl get secret db-secret -o jsonpath='{.data.DB_USER}' | base64 -d`

```
type: Opaque
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl get secret db-secret -o jsonpath='{.data.DB_USER}' | base64 -d
adminu1@ubuntu-desktop1:~/k8s_labs/config$
```

**e** Explain: why is base64 NOT the same as encryption?

**Verify with:**

```
kubectl get secret
kubectl describe secret db-secret
kubectl get secret db-secret -o yaml
kubectl get secret db-secret -o jsonpath='{.data.DB_USER}' | base64 -d
```

How are values stored in the Secret YAML? Why

is base64 NOT encryption?

1- Values are stored in base64 encoded format inside the data field

2- **Base64 is only encoding, not encryption. Anyone can easily decode it back to the original value**

## 5 Inject Secret as Environment Variables

Medium ☆☆

Create a Pod that injects the **db-secret** values as environment variables and verify they are decoded automatically.

| Field | Value |
|---|---|
| File to apply | `06-secret-as-envvars.yaml` |
| Pod Name | `db-pod` |
| Image | `busybox` |
| Injection method | `envFrom → secretRef: db-secret` |
| Command | `echo User=$DB_USER && echo DB=$DB_NAME && sleep 3600` |

### Requirements:

**a** Apply `06-secret-as-envvars.yaml`

```
u1@ubuntu-desktop1:~/k8s_labs/config$ nano 06-secret-as-envvars.yaml
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl apply -f 06-secret-as-envvars.yaml
pod/db-pod created
```

**b** Run `kubectl logs db-pod` — are the values decoded?

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl logs db-pod
User=admin
DB=myapp
```

**c** Run `kubectl describe pod db-pod` — can you see the Secret values there?

```
u1@ubuntu-desktop1:~$ kubectl describe pod db-pod
Name:             db-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 15 Mar 2026 12:39:21 +0200
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.63
IPs:
  IP:  10.244.0.63
Containers:
  db-container:
    Container ID:  docker://cd2e25ba99352ab2862bc22e97e3078b6033f45d0aa1bf67d1818545de21e07d
    Image:         busybox
    Image ID:      docker-pullable://busybox@sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4207dd76f
    Port:          <none>
    Host Port:     <none>
    Command:
      sh
      -c
      echo User=$DB_USER && echo DB=$DB_NAME && sleep 3600
    State:          Running
      Started:      Sun, 15 Mar 2026 12:39:24 +0200
    Ready:          True
    Restart Count:  0
    Environment Variables from:
      db-secret   Secret  Optional: false
    Environment:  <none>
```

**Verify with:**

```
kubectl logs db-pod
kubectl exec -it db-pod -- env | grep DB
kubectl describe pod db-pod
```

Are values decoded in logs? What did you see?

Are values visible in kubectl describe pod?

**1-Yes. The Secret values appear decoded inside the container logs.**

**2- No. The Secret values are not visible; Kubernetes only shows that the values come from a Secret.**

# Part 3 — Combined Pattern

20 pts

## 6 Inject ConfigMap + Secret Together

Medium ☆☆

Create a Pod that uses **both** app-config (ConfigMap) and db-secret (Secret) — injecting both into the same container.

| Field | Value |
|---|---|
| File to apply | `08-combined-cm-and-secret.yaml` |
| Pod Name | `fullapp-pod` |
| Image | `busybox` |
| ConfigMap inject | `envFrom → configMapRef: app-config` |
| Secret inject | `envFrom → secretRef: db-secret` |

### Requirements:

**a** Apply `08-combined-cm-and-secret.yaml`

```
u1@ubuntu-desktop1:~$ nano 08-combined-cm-and-secret.yaml
u1@ubuntu-desktop1:~$ kubectl apply -f 08-combined-cm-and-secret.yaml
pod/fullapp-pod created
```

**b** Run `kubectl logs fullapp-pod` — verify both sections print correctly

```
u1@ubuntu-desktop1:~$ kubectl logs fullapp-pod
DB_NAME=myapp
APP_PORT=8080
WEBAPP_SVC_SERVICE_HOST=10.107.38.171
WEBAPP_SVC_SERVICE_PORT=80
WEBAPP_SVC_PORT=tcp://10.107.38.171:80
WEBAPP_SVC_PORT_80_TCP_ADDR=10.107.38.171
WEBAPP_SVC_PORT_80_TCP_PORT=80
WEBAPP_SVC_PORT_80_TCP_PROTO=tcp
APP_COLOR=blue
APP_ENV=production
DB_PASSWORD=p@ssw0rd123
WEBAPP_SVC_PORT_80_TCP=tcp://10.107.38.171:80
```

**c** Run `kubectl exec -it fullapp-pod -- env | sort` — count how many env vars come from ConfigMap vs Secret

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl exec webapp-configured-68fcfdb595-6n4nr -- env | grep -E 'APP_ENV|DB_USER'
APP_ENV=production
DB_USER=admin
u1@ubuntu-desktop1:~/k8s_labs/config$
```

```
u1@ubuntu-desktop1:~$ kubectl exec -it fullapp-pod -- env | sort
E0315 13:01:48.211481  602361 runtime.go:142] "Observed a panic" panic="runtime error: invalid memory address or nil pointer dereference" panicGoValue="\"inva
id memory address or nil pointer dereference\"" stacktrace=<
                                                                goroutine 75 [running]:
                                                                                        k8s.io/apimachinery/pkg/util/runtime.logPanic({0x289ca38, 0x3c0f0a0},
0x2008260, 0x3bb6560})
                                k8s.io/apimachinery/pkg/util/runtime/runtime.go:132 +0xbc
                                                                                                k8s.io/apimachinery/pkg/util/runtime.handleCrash({0x289ca38, 6
3c0f0a0}, {0x2008260, 0x3bb6560}, {0xc000889da8, 0x0, 0x0?})
                                                                        k8s.io/apimachinery/pkg/util/runtime/runtime.go:107 +0x116
                                                                                                                                        k8s.io/apimachinery/pk
/util/runtime.HandleCrash({0x0, 0x0, 0xc0004a96c0?})
                                                                k8s.io/apimachinery/pkg/util/runtime/runtime.go:64 +0x17b
                                                                                                                        panic({0x2008260?, 0x3bb6560?}

untime/panic.go:783 +0x132
                                k8s.io/kubectl/pkg/cmd/exec.(*terminalSizeQueueAdapter).Next(0x0?)
                                                                                                        k8s.io/kubectl/pkg/cmd/exec/exec.go:414 +0x15
```

**d** Explain: which vars should go in ConfigMap and which in Secret?

**ConfigMap vars (non-sensitive)**

**APP_ENV**

**APP_PORT**

**APP_COLOR**

**MAX_CONNECTIONS**

**Secret vars (sensitive)**

**DB_USER**

**DB_PASSWORD**

**DB_NAME**

## 7 Deployment with ConfigMap + Secret

**Hard** ☆☆☆

Apply the Deployment that uses both ConfigMap and Secret. Verify that **all 3 pods** automatically get the same configuration.

## % ConfigMap & Secret — Lab



| Field | Value |
|---|---|
| File to apply | `09-deployment-with-config.yaml` |
| Deployment Name | `webapp-configured` |
| Replicas | `3` |
| ConfigMap key used | `APP_ENV` |
| Secret keys used | `DB_USER, DB_PASSWORD` |

### Requirements:

**a** Apply `09-deployment-with-config.yaml`

```
u1@ubuntu-desktop1:~/k8s_labs/config$ nano 09-deployment-with-config.yaml
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl apply -f 09-deployment-with-config.yaml
deployment.apps/webapp-configured created
```

**b** Confirm all 3 pods are Running: `kubectl get pods -l app=webapp-configured`

```
u1@ubuntu-desktop1:~/k8s_labs/config$ kubectl get pods -l app=webapp-configured
NAME                                READY   STATUS    RESTARTS   AGE
webapp-configured-68fcfdb595-58tzq  1/1     Running   0          19s
webapp-configured-68fcfdb595-6n4nr  1/1     Running   0          19s
webapp-configured-68fcfdb595-cc8gt  1/1     Running   0          19s
```

**c** Pick any pod and verify it has the env vars: `kubectl exec -it <pod> -- env | grep -E 'APP_ENV|DB_USER'`

**d** Pick a DIFFERENT pod and run the same check — do all pods have the same vars?

**e** Explain: what is the advantage of using ConfigMap + Secret with a Deployment vs hardcoding values in the YAML?

**Using ConfigMap and Secret allows separating configuration from the application. This makes it easier to update configuration without rebuilding the container image and allows multiple pods to share the same configuration securely.**